

Rviz Visualization

我建议 RViz 可视化按一个原则设计：**不是把所有 debug 数据都画出来，而是让你一眼回答四个问题：**

代码块

- 1 1. 现在风险场在哪里？
- 2 2. 当前轨迹为什么被认为安全/不安全？
- 3 3. planner 是不是因为完整性风险改变了轨迹？
- 4 4. 如果触发 replan/emergency，是 P0/P5/P1/P2/P3/P4 哪一层造成的？

你的 Safety Planner 已经分成 P0 risk field、P5 hard gate、P1 soft cost、P2 candidate ranking、P3 reference bias、P4 A* fallback。RViz 可视化也应该严格沿这个分层来做，不要混在一个 marker 里。P5 是唯一 hard safety gate，P1/P2/P3/P4 只是 preference / ranking / bias / fallback，这个边界在可视化里也要体现出来。

总体推荐：做 7 组 RViz 图层

最终建议可视化这些图层：

层	名称	作用	默认显示
V0	RiskGridMap health	看 P0 是否 ready/stale/unknown	开
V1	Predicted PL field	看未来预测 PL 空间分布	开
V2	Current + future trajectory IM	看当前执行轨迹哪里违反 PL < AL	开
V3	P5 gate decision	看为什么触发 replan/emergency	开
V4	P1 integrity gradient/cost samples	看优化器是否真的被 risk cost 推动	调试开
V5	P2/P3/P4 planner decision overlays	看候选、reference bias、A* guide 是否生效	调试开
V6	Source/ validity/ unknown/stale map	看 risk field 是否可信	调试开

不要一开始就画太多复杂 3D 体素。建议先实现 **V1 + V2 + V3**，这三层最有用。

V0: RiskGridMap Health 面板

要显示什么

P0 是否工作正常：

代码块

```
1 generation_id
2 age_s
3 valid_ratio
4 unknown_ratio
5 stale
6 horizons_s
7 current selected horizon
```

怎么显示

用 `visualization_msgs/msg/MarkerArray` 的 `TEXT_VIEW_FACING`，放在地图左上角。

Topic:

代码块

```
1 /iap/rviz/risk_grid_health
```

Marker 内容示例:

代码块

```
1 RiskGridMap
2 gen: 1482
3 age: 0.21 s
4 valid: 82.4%
5 unknown: 12.7%
6 horizons: [0.0, 0.5, 1.0, 1.5, 2.0]
7 status: READY
```

颜色:

代码块

```
1 READY: green
2 STALE: yellow
3 NOT_READY / UNKNOWN_HIGH: red
```

这个层很重要，因为很多 planner 问题其实不是优化器问题，而是 P0 没 ready、unknown 太多或 generation 没更新。

V1: Predicted PL Field

这是最核心的可视化。

要显示什么

显示 P0 multi-horizon field 中每个 voxel 的预测风险:

代码块

```
1 hpl_pred
2 vpl_pred
```

```
3   pl_scalar = max(hpl_pred, vpl_pred)
4   c_pi
5   valid / unknown / stale
```

但 RViz 不建议同时显示所有 horizon。应该允许选择一个 horizon：

代码块

```
1   rviz:
2     selected_horizon_s: 1.0
```

推荐显示方式

方式 A: PointCloud2 彩色点云，推荐默认

Topic:

代码块

```
1   /iap/rviz/predicted_pl_cloud
```

Message:

代码块

```
1   sensor_msgs/msg/PointCloud2
```

每个 voxel center 一个点，字段：

代码块

```
1   x, y, z
2   rgb
3   pl
4   hpl
5   vpl
6   c_pi
7   valid
8   unknown
9   source_flags
```

颜色映射：

代码块

```
1 blue / cyan: low PL
2 green: acceptable PL
3 yellow: medium PL
4 orange: high PL
5 red: very high PL
6 gray: unknown
7 dark gray / transparent: stale
```

建议只显示水平 slice，例如当前 UAV 高度附近：

代码块

```
1 pl_cloud:
2     z_slice_mode: current_altitude
3     z_slice_half_thickness_m: 0.75
```

否则 3D 体素太密，RViz 会非常乱。

V2: Trajectory Integrity Evaluation

这是第二核心图层，用来回答：**当前轨迹哪里安全，哪里不安全。**

要显示什么

沿当前 B-spline 未来 2 秒采样，每个 sample 显示：

代码块

```
1 position p(t)
2 query_time = now + tau
3 HPL_pred
4 VPL_pred
5 HAL
6 VAL
7 IM_H = HAL - HPL
8 IM_V = VAL - VPL
9 IM_min = min(IM_H, IM_V)
10 sample_class = GOOD / BAD / UNKNOWN
```

怎么显示

Topic:

```
代码块 /iap/rviz/trajectory_integrity_samples
```

Message:

代码块

```
1 visualization_msgs/msg/MarkerArray
```

每个 sample 一个 sphere:

代码块

```
1 GOOD: green sphere
2 BAD: red sphere
3 UNKNOWN: gray sphere
4 STALE: yellow sphere
```

Marker 大小根据严重程度变化:

代码块

```
1 radius = base_radius + k * max(0, -IM_min)
```

再加一条轨迹线:

代码块

```
1 /iap/rviz/current_traj_integrity_colored
```

用 `LINE_LIST` 或多个 `LINE_STRIP` 分段显示:

代码块

```
1 green segment: IM >= margin
2 yellow segment: 0 <= IM < margin
3 red segment: IM < 0
4 gray segment: unknown
```

Text label

对 first bad sample 加一个文字:

代码块

```
1 first_bad_tau = 0.8 s
2 IM = -0.42 m
3 reason = future_bad
```

Topic:

代码块

```
1 /iap/rviz/p5_first_bad_label
```

这比只看日志直观很多。

V3: P5 Gate Decision 可视化

P5 是 hard gate，所以它需要最清晰的可视化。

要显示什么

P5 当前 action:

代码块

```
1 OK
2 REQUEST_REPLAN
3 REQUEST_EMERGENCY_STOP_CANDIDATE
```

以及原因:

代码块

```
1 current_invalid
2 current_stale
3 current_low_margin
4 future_bad
5 future_unknown
6 al_invalid
7 final_gate_failed
```

怎么显示

1. 无人机上方大状态牌

Topic:

代码块

```
1 /iap/rviz/p5_gate_status
```

Marker:

代码块

```
1 TEXT_VIEW_FACING
```

内容:

代码块

```
1 P5: REQUEST_REPLAN
2 reason: future_bad
3 future_min_IM: -0.31 m
4 first_bad_tau: 1.2 s
5 bad_ratio: 0.35
6 unknown_ratio: 0.05
```

颜色:

代码块

```
1 OK: green
2 REQUEST_REPLAN: orange
3 EMERGENCY_CANDIDATE: red
4 DISABLED: gray
```

2. 当前点 IM 柱状条

可以在 UAV 旁边画两个竖直 bar:

代码块

```
1 H-IM = HAL - HPL
2 V-IM = VAL - VPL
```


Marker:

代码块

```
1 CUBE
```

颜色:

代码块

```
1 positive: green
2 near zero: yellow
3 negative: red
```

这对解释“当前 ARAIM 是否已经报警”很有用。

V4: P1 Soft Cost / Gradient 可视化

P1 用于回答: 后端优化器有没有真的感受到 risk field?

这层默认关闭, 只在调试 P1 时打开。

要显示什么

沿优化中的 B-spline sample 显示:

代码块

```
1 sample position
2 c_pi
3 grad_c_pi
4 hit / miss / stale
5 weighted gradient
```

怎么显示

Topic:

代码块

```
1 /iap/rviz/p1_integrity_samples
2 /iap/rviz/p1_integrity_gradients
```

Marker:

代码块

- 1 SPHERE_LIST for samples
- 2 ARROW for gradients

颜色:

代码块

- 1 low c_pi: blue/green
- 2 high c_pi: orange/red
- 3 miss/stale: gray

箭头方向:

代码块

- 1 grad_c_pi direction

注意: 优化器是最小化 cost, 所以轨迹被推向的是 `-grad_c_pi` 方向。为了直观, 可以画两种箭头之一:

代码块

- 1 grad arrow: 指向 cost 上升方向
- 2 avoidance arrow: 指向 `-grad`, 表示 planner 被推动方向

我建议画 `-grad_c_pi`, 命名为:

代码块

- 1 /iap/rviz/p1_integrity_push_vectors

这样更容易解释。

额外文字

显示 P1 当前调参核心指标:

代码块

```
1 grad_ratio = weighted_grad_integrity / grad_original
2 target = 5% - 20%
```

如果 `grad_ratio < 1%`，说明 P1 太弱；如果 `> 30%`，说明可能压过 collision / feasibility。

V5: P2 / P3 / P4 Planner Decision Overlay

这层用来证明 planner 的 decision chain 是否按预期生效。

V5.1 P2 Candidate Ranking

Topic:

代码块

```
1 /iap/rviz/p2_candidate_trajectories
```

显示多条 candidate trajectory:

代码块

```
1 original winner
2 P2 winner
3 rejected candidates
```

Marker:

代码块

```
1 LINE_STRIP
```

颜色建议:

代码块

```
1 original winner: blue
2 P2 selected winner: purple / magenta
3 rejected candidates: gray transparent
4 P5 final gate rejected: red dashed or red markers
```

每条候选加文字：

代码块

```
1 cand 3
2 orig_cost: 0.42
3 risk_score: 1.83
4 unknown: 4%
5 selected: yes
```

这能直接展示 P2 是否只是 metrics-only，还是实际改变了 winner。

V5.2 P3 Reference Bias

Topic:

代码块

```
1 /iap/rviz/p3_reference_bias
```

显示三类东西：

代码块

```
1 nominal global reference
2 biased global reference
3 nominal local target
4 biased local target
```

Marker:

代码块

```
1 LINE_STRIP for reference
2 SPHERE for target
3 ARROW from nominal target to biased target
```

颜色：

代码块

```
1 nominal reference: blue
2 biased reference: purple
```

```
3 nominal local target: white
4 biased local target: orange
5 fallback/no bias: gray
```

文字 label:

代码块

```
1 P3-local used
2 score improvement: 12.4%
3 reason: lower risk target
```

或者:

代码块

```
1 P3-global disabled
2 reason: corridor coverage insufficient
3 valid_ratio: 0.42 < 0.8
```

这个很关键，因为 P3 很容易被误解成 global planner。可视化中必须显示 coverage gate。

V5.3 P4 Risk-aware A* Guide

Topic:

代码块

```
1 /iap/rviz/p4_astar_guides
```

显示:

代码块

```
1 original A* guide path
2 risk-aware A* guide path
3 selected guide path
4 collision segment start/end
```

颜色:

```
1 original A*: blue
2 risk-aware A*: orange
3 selected: thick green
4 fallback original: yellow
5 rejected risk path: red
```

文字：

代码块

```
1 path_ratio: 1.42 > 1.30
2 fallback: original
```

由于 P4 只在 collision segment guide 中生效，建议同时显示 collision segment：

代码块

```
1 collision segment control points: red spheres
```

这样你能一眼确认：P4 没运行到底是 bug，还是因为没有 collision segment。

V6: Validity / Source / Unknown / Stale 可视化

这层用于 debug P0 数据质量。

要显示什么

不同 voxel 的状态：

代码块

```
1 valid
2 unknown
3 stale
4 fallback
5 source_flags
```

source_flags 可以区分：

代码块

```
1 GNSS only
```

- 2 LiDAR only
- 3 Fusion
- 4 None

怎么显示

Topic:

代码块

- 1 /iap/rviz/risk_validity_cloud

颜色:

代码块

- 1 Fusion: green
- 2 GNSS: blue
- 3 LiDAR: cyan
- 4 None/unknown: gray
- 5 Fallback: yellow
- 6 Stale: dark gray

可以和 V1 分开，不要默认同时显示，否则太乱。

推荐 ROS Topic 总表

代码块

1	/iap/rviz/risk_grid_health	MarkerArray / Text
2	/iap/rviz/predicted_pl_cloud	PointCloud2
3	/iap/rviz/predicted_pl_voxels	MarkerArray, optional
4	/iap/rviz/trajectory_integrity_samples	MarkerArray
5	/iap/rviz/current_traj_integrity_colored	MarkerArray
6	/iap/rviz/p5_gate_status	MarkerArray
7	/iap/rviz/p1_integrity_samples	MarkerArray
8	/iap/rviz/p1_integrity_push_vectors	MarkerArray
9	/iap/rviz/p2_candidate_trajectories	MarkerArray
10	/iap/rviz/p3_reference_bias	MarkerArray
11	/iap/rviz/p4_astar_guides	MarkerArray
12	/iap/rviz/risk_validity_cloud	PointCloud2

如果只做最小可用版本，我建议先做这 5 个：

代码块

```
1 /iap/rviz/predicted_pl_cloud
2 /iap/rviz/current_traj_integrity_colored
3 /iap/rviz/trajectory_integrity_samples
4 /iap/rviz/p5_gate_status
5 /iap/rviz/risk_grid_health
```

推荐颜色语义

全系统统一颜色，不要每个模块自己定义。

语义	颜色
safe / valid / OK	green
low risk	blue / cyan
marginal	yellow
high risk	orange
violation / emergency	red
unknown	gray
stale	dark gray
fallback	purple
disabled	transparent gray

PL field 可以用连续 colormap：

代码块

```
1 0 ~ low_threshold: blue
2 low ~ replan_margin: green
3 near AL: yellow
4 PL > AL: red
5 unknown: gray
```

但是注意：P0 自己不算 AL/IM，所以 PL field 的颜色最好只是 PL magnitude；只有 V2/P5 trajectory layer 才显示 `PL < AL` 的 hard safety 结果。

推荐实现结构

不要把 RViz 可视化逻辑塞进 P0/P1/P5 核心类。建议新建一个独立可视化类：

代码块

```
1  class SafetyPlannerRvizVisualizer {
2  public:
3      void configure(const RvizVizParams& params);
4
5      void publishRiskGrid(
6          const std::shared_ptr<const RiskGridSnapshot>& snapshot,
7          double selected_horizon_s);
8
9      void publishTrajectoryIntegrity(
10         const TrajectoryIntegrityReport& report);
11
12     void publishP5Status(
13         const PlannerIntegrityStatus& status);
14
15     void publishP1Debug(
16         const P1IntegrityDebugFrame& debug);
17
18     void publishP2Candidates(
19         const P2CandidateRankingDebug& debug);
20
21     void publishP3ReferenceBias(
22         const P3ReferenceBiasResult& result);
23
24     void publishP4AstarGuides(
25         const P4AstarDebug& debug);
26 };
```

参数：

代码块

```
1  rviz_safety:
2      enable: true
3      publish_rate_hz: 5.0
4
5      show_pl_cloud: true
6      show_voxels: false
7      show_trajectory_im: true
8      show_p5_status: true
9      show_p1_debug: false
10     show_p2_candidates: false
```

```
11     show_p3_bias: false
12     show_p4_astar: false
13     show_validity_cloud: false
14
15     selected_horizon_s: 1.0
16     z_slice_mode: current_altitude
17     z_slice_half_thickness_m: 0.75
18
19     max_visualized_voxels: 5000
20     marker_lifetime_s: 0.3
21     alpha_voxel: 0.35
22     alpha_candidate: 0.45
```

最有价值的一张 RViz 画面应该长什么样

我建议你的主 RViz layout 是：

代码块

1. 原始 occupancy / inflated map
2. UAV 当前位姿
3. 当前执行 B-spline
4. 未来 2s trajectory integrity colored samples
5. selected horizon 的 predicted PL field slice
6. P5 status text
7. first_bad_tau label

这样导师或自己 debug 时可以一眼看到：

代码块

- 1 红色 risk 区域在哪里；
- 2 当前轨迹是否穿过高 PL 区域；
- 3 P5 是否因为 future_bad 触发 replan；
- 4 first bad point 离当前还有多久；
- 5 unknown 是否太多。

这比画一堆算法内部变量更有价值。

RViz 可视化开发顺序

Phase 1: 核心运行可视化

代码块

- 1 1. RiskGrid health text
- 2 2. Predicted PL cloud / slice
- 3 3. Current trajectory IM colored line
- 4 4. P5 status text
- 5 5. First bad sample marker

这一步完成后，你已经能 debug 80% 的问题。

Phase 2: 优化器行为可视化

代码块

- 1 6. P1 sampled cost points
- 2 7. P1 push vectors
- 3 8. grad_ratio text

用于解释“为什么轨迹往这边偏”。

Phase 3: 多候选和 reference 可视化

代码块

- 1 9. P2 candidate trajectories
- 2 10. P3 nominal vs biased target/reference

用于实验展示 planner 是否主动选择低风险路径。

Phase 4: A* fallback 可视化

代码块

- 1 11. P4 original/risk-aware/selected A* guide
- 2 12. path length ratio label

只在 P4 实验时打开。

不建议可视化什么

这些不建议放 RViz 主界面：

代码块

1. 每个 voxel 的完整 GNSS/LiDAR FIM 矩阵
2. 每个 hypothesis 的 ARAIM PL 分解
3. 每次 L-BFGS iteration 的全部 gradient
4. 所有 horizon 同时显示
5. 所有 candidate + all samples + all vectors 同时显示

这些适合 CSV、plot 或离线分析，不适合 RViz。RViz 应该用于空间因果关系，不是用于看所有数值。

最终建议

你应该把 RViz 可视化设计成三层：

代码块

- 1 Safety layer:
2 P5 status + trajectory IM + first_bad point
3
- 4 Risk field layer:
5 P0 predicted PL slice + valid/unknown/stale map
6
- 7 Planner behavior layer:
8 P1 gradient / P2 candidates / P3 bias / P4 A* guide

最先实现：

代码块

- 1 predicted_pl_cloud
- 2 current_traj_integrity_colored
- 3 trajectory_integrity_samples
- 4 p5_gate_status
- 5 risk_grid_health

这样你就能非常清楚地展示 Safety Planner 的核心贡献：**不是只在当前点报警，而是在未来轨迹上可视化 predicted PL、AL/IM、安全边界和 planner 反应。**

